

TypeScript vs JavaScript

Feature	JavaScript	TypeScript
Definition	A scripting language used to create dynamic web applications.	A superset of JavaScript developed by Microsoft that adds static typing.
Typing	Dynamically typed	Statically typed
Type Checking	Happens at runtime	Happens during compilation
Compilation	No compilation required	Compiles to JavaScript
Error Detection	Errors found during execution	Many errors caught before execution
Variable Types	No explicit types	Supports explicit types
IDE Support	Basic autocomplete	Advanced IntelliSense, refactoring, and type checking
Interfaces	Not available	Supported
Generics	Not available	Supported
Enums	Not available	Supported
Access Modifiers	Limited support	Supports public, private, protected
Code Maintainability	Harder for large projects	Easier for large projects
Learning Curve	Easier to learn	Slightly steeper due to type system
Performance	Executes directly	Compiles to JavaScript before execution
Scalability	Suitable for small-medium projects	Excellent for large enterprise applications
Tooling	Good	Excellent
Refactoring Support	Limited	Strong support
OOP Features	Basic	Advanced

Feature	JavaScript	TypeScript
Used By	Frontend and backend development	Large-scale frontend and backend development

Example 1: Variable Declaration

JavaScript

```
let age = 25;
```

```
age = "Twenty Five";
```

```
console.log(age);
```

✅ Runs successfully

Output:

Twenty Five

TypeScript

```
let age: number = 25;
```

```
age = "Twenty Five";
```

❌ Compile-time error

Type 'string' is not assignable to type 'number'

Example 2: Function Parameters

JavaScript

```
function add(a, b) {  
    return a + b;  
}
```

```
console.log(add(10, 20));
```

```
console.log(add("10", 20));
```

Output:

30

1020

TypeScript

```
function add(a: number, b: number): number {  
    return a + b;  
}
```

```
add("10", 20);
```

 Error

Argument of type 'string' is not assignable to parameter of type 'number'

Example 3: Interfaces

JavaScript

```
const user = {  
    id: 1,  
    name: "John"  
};
```

No structure enforcement.

TypeScript

```
interface User {  
    id: number;  
    name: string;  
}
```

```
const user: User = {  
  id: 1,  
  name: "John"  
};
```

Ensures object structure is correct.

Example 4: Enums

JavaScript

```
const Status = {  
  ACTIVE: "ACTIVE",  
  INACTIVE: "INACTIVE"  
};
```

TypeScript

```
enum Status {  
  ACTIVE,  
  INACTIVE  
}
```

```
let status: Status = Status.ACTIVE;
```

Example 5: Generics

JavaScript

```
function getData(data) {  
  return data;  
}
```

TypeScript

```
function getData<T>(data: T): T {  
    return data;  
}
```

```
const name = getData<string>("Bala");  
const age = getData<number>(30);
```

Why Playwright Projects Prefer TypeScript

Benefit	Example
Early error detection	Catches locator and type mistakes during development
Better IntelliSense	Suggestions for Page, Locator, BrowserContext, etc.
Refactoring support	Rename methods safely across framework
Maintainability	Easier for large automation suites
Strong typing	Better Page Object Models and utility classes

Playwright Example

```
import { test, Page } from '@playwright/test';
```

```
async function login(page: Page) {  
    await page.fill('#username', 'admin');  
    await page.fill('#password', 'admin123');  
    await page.click('#login');  
}
```

```
test('login test', async ({ page }) => {  
    await login(page);  
});
```

The Page type provides IntelliSense and compile-time validation, making Playwright automation more reliable.

Summary

Use JavaScript When	Use TypeScript When
Learning web basics	Building automation frameworks
Small scripts	Enterprise projects
Quick prototypes	Long-term maintainable code
No type safety needed	Type safety is important

For Playwright automation frameworks, TypeScript is generally the preferred choice because it improves code quality, maintainability, and developer productivity.